

02-SRS-Software-Requirements-Specification

SRS — Software Requirements Specification

Leish! v2 — Beauty Booking Marketplace

Table of Contents

1. Introduction
2. Overall Description
3. System Features (Summary)
4. External Interface Requirements
5. System Features — Detailed (SRS-F)
6. Data Requirements
7. Other Requirements
8. Verification & Validation
9. Appendices

SRS — Software Requirements Specification

Leish! v2 — Beauty Booking Marketplace

FIELD	VALUE
Document ID	LEISH-SRS-v2.0
Version	2.0.0
Status	Baseline
Date	2026-08-29
Complies With	IEEE Std 830-1998 (SRS)
Predecessor	BRS (01)
Successor	FRS (03), NFR (04)
Repo	<code>src/</code> (Next.js 16 App Router, React 19, TypeScript)

Table of Contents

- [1. Introduction](#)
- [2. Overall Description](#)
- [3. System Features \(Summary\)](#)
- [4. External Interface Requirements](#)
- [5. System Features — Detailed \(SRS-F\)](#)
- [6. Data Requirements](#)
- [7. Other Requirements](#)
- [8. Verification & Validation](#)
- [9. Appendices](#)

1. Introduction

1.1 PURPOSE

This SRS translates the business requirements (BRS [01](#)) into verifiable software requirements. Its audience is architects, developers, QA, and auditors. It is the single reference for what the system **shall** do, under what constraints, and how compliance will be shown.

1.2 SCOPE

The system is **Leish! v2** — a web platform ([leish_v2](#)) that provides:

- Public catalog (artists/studios) with faceted discovery.
- Authenticated booking request → quotation → two-step payment → completion → review.
- Real-time per-booking messaging.
- Admin operations with audit trail and platform settings.
- Transactional email and observability.

Out of scope is listed in BRS §4.2 and restated in §2.5 below.

1.3 DEFINITIONS, ACRONYMS, ABBREVIATIONS

TERM	MEANING
MUA	Makeup Artist
MYR/sen	Ringgit; amounts stored as integer sen (1 MYR = 100 sen)
Billplz	Malaysian payment gateway (Billplz API v3)
DFA	DbFacade (src/server/db.ts) — PG or SQLite behind same async API
JWT/JTI	JSON Web Token / JWT ID (revocation key)
SSE	Server-Sent Events (chat live stream)
PDPA	Malaysia Personal Data Protection Act 2010
RQ	Requirement

See full glossary in [docs/09-Glossary.md](#).

1.4 REFERENCES

- BRS [01-BRS-Business-Requirements-Specification.md](#)
- [AGENTS.md](#) (project conventions), [docs/ARCHITECTURE.md](#), [docs/PDPA_RETENTION_GUIDELINES.md](#)
- Source of truth code: [src/server/db.ts](#), [src/server/session.ts](#), [src/server/bookings.ts](#), [src/server/payments.ts](#), [src/server/payouts.ts](#), [src/server/quotations.ts](#), [src/server/catalog.ts](#), [src/server/validation.ts](#)
- Standards: IEEE 830, OWASP ASVS L2, WCAG 2.1 AA

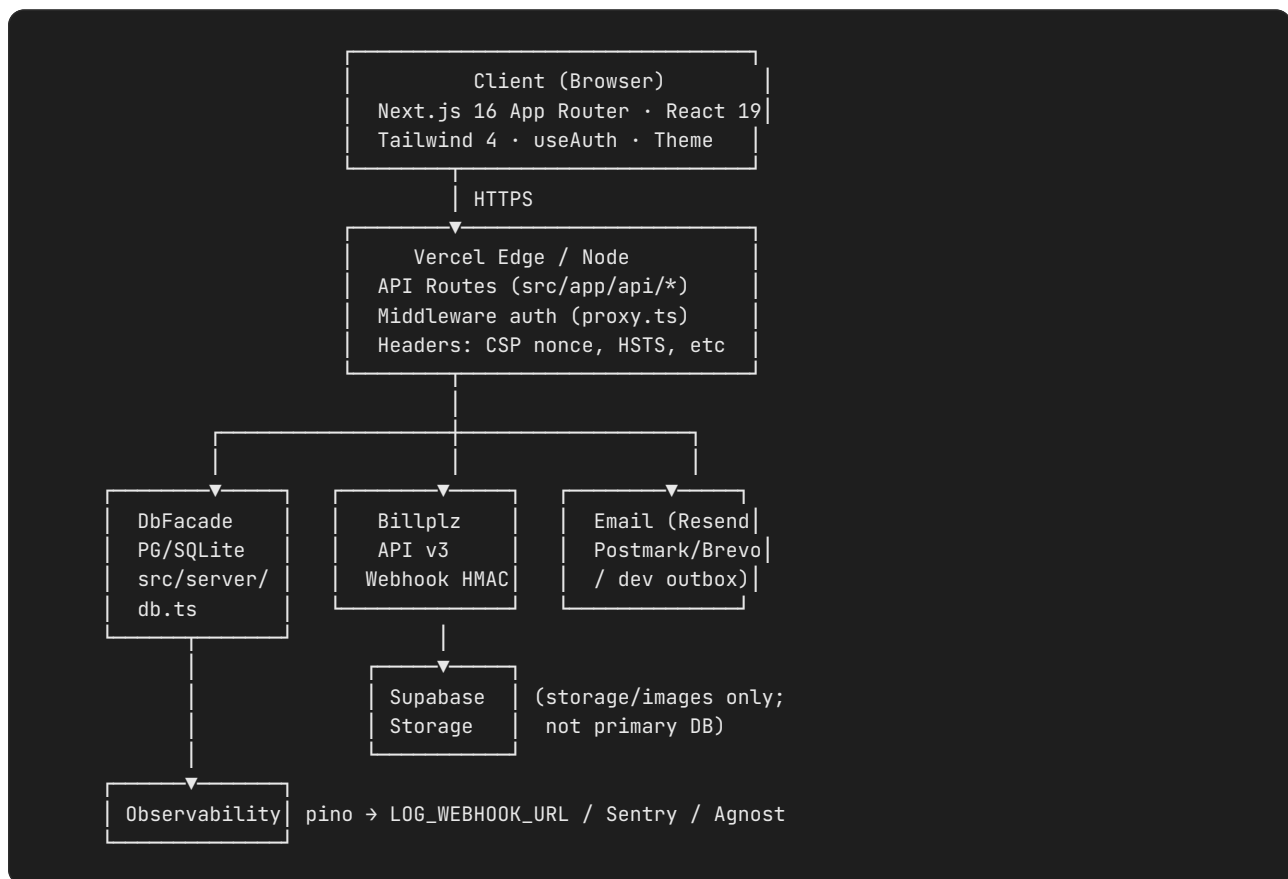
1.5 OVERVIEW

§2 gives the product context; §3 summarizes features; §4 defines interfaces; §5 decomposes each feature into verifiable SRS requirements with IDs; §6 details data; §7 covers adoption; §8 maps verification.

Conformance language: **shall** = mandatory, **should** = recommended, **may** = optional.

2. Overall Description

2.1 PRODUCT PERSPECTIVE



- **Single persistence:** `getDb()` facade (`src/server/db.ts:828`). `isPostgres()` switches between `pg.Pool` (production) and `node:sqlite DatabaseSync` (dev/tests). Schema is defined once per backend (`PG_SCHEMA` / `SQLITE_SCHEMA`) and kept in sync by `extractSchemaTables()` drift detection.
- **Stateless auth:** HS256 JWT (`jose`) signed with `SESSION_SECRET` , stored in `leish_session` httpOnly cookie, `SameSite=lax` , `Secure` in prod, 7-day TTL, JTI revocation table `sessions` .
- **Deployment:** Vercel preferred; Docker `standalone` output also supported; health at `GET /api/health` .

2.2 PRODUCT FUNCTIONS (SUMMARY)

1. Catalog browsing & search (guests and authed users).
2. Account lifecycle (register, login, verify, reset, consent).
3. Booking lifecycle with slot locking.
4. Quotation authoring & expiry.
5. Hybrid payments (deposit + balance) via Billplz/dev.
6. Payout & commission settlement.
7. Reviews (gated, atomic).
8. Real-time chat.
9. Admin CRUD + audit + settings.
10. Notifications, logging, analytics.

FRS (`03`) expands each function into `FR-xxx` with acceptance criteria.

2.3 USER CLASSES AND CHARACTERISTICS

CLASS	COUNT (LAUNCH)	TECHNICAL SKILL	FREQUENCY	PRIVILEGES
Guest	Unbounded	Low	Occasional	Read catalog, register
Customer	~500	Low	1–4/yr	Book, pay, chat, review, export/delete
Artist	~10–30	Low–Med	Daily (dashboard)	Claim profile, manage bookings, quote, complete
Studio	~4–10	Med	Daily	Same as artist (studio_id scoping)
Admin	2–5	High	Daily	Full CRUD + audit + settings + payouts
Cron/ System	—	—	Scheduled	Sweep expired quotes, email retries

Accessibility: WCAG 2.1 AA, responsive (mobile-first), dark/light theme persisted in `localStorage` (`src/lib/theme.tsx`).

2.4 OPERATING ENVIRONMENT

LAYER	REQUIREMENT
Runtime	Node ≥ 22 (for <code>node:sqlite</code>), Next.js 16.3.1, React 19.2.8
Hosting	Vercel (primary) or Docker <code>node:20-alpine</code> with <code>output:"standalone"</code>
DB — Prod	PostgreSQL (Neon/Supabase pooler) via <code>DATABASE_URL</code>
DB — Dev/Test	<code>node:sqlite</code> (<code>LEISH_DB_PATH</code> or <code>:memory:</code>)
Browsers	Evergreen Chrome/Firefox/Safari/Edge (last 2 versions)
Env	<code>.env.local</code> (prod), <code>.env.example</code> template; <code>src/env.ts</code> validates <code>SESSION_SECRET</code> in prod

2.5 DESIGN AND IMPLEMENTATION CONSTRAINTS

- Shall use **Next.js App Router** (no Pages Router).
- Shall not commit secrets — use Vercel/CI secrets; `SESSION_SECRET` must be 32 random bytes base64 (`openssl rand -base64 32`).
- Shall keep `PG_SCHEMA` and `SQLITE_SCHEMA` in sync (drift warning via `detectSchemaDrift()`).
- Shall never store money as float — `INTEGER sen` only.
- Shall not trust client-sent amounts — server computes `bookings.price` and `quotations.total` .
- Root `loading.tsx` is **forbidden** (breaks `notFound()` 404 semantics — see AGENTS.md §11).
- Absolute imports use `@/` alias to `./src/` .

2.6 ASSUMPTIONS AND DEPENDENCIES

See BRS §10.2/10.3. Additions:

- Assumes `jose` 6.2.8, `pg` 8.23.0, `pino` 10.3.1 remain compatible.
- Depends on Billplz collection setup (`BILLPLZ_COLLECTION_ID`) before live payments.
- Depends on Supabase project for image hosting; graceful fallback to `/images/*` placeholders if unavailable.

3. System Features (Summary)

FEATURE ID	FEATURE	PRIORITY	BRS TRACE
F-01	Authentication & Session	Must	BR-06..BR-10
F-02	Catalog (Artists/Studios)	Must	BR-01..BR-05
F-03	Booking Management	Must	BR-11..BR-13, BR-15
F-04	Quotations	Must	BR-14..BR-15
F-05	Payments (Hybrid)	Must	BR-16..BR-19, BR-22
F-06	Payouts & Commission	Must	BR-20..BR-21
F-07	Reviews & Ratings	Must	BR-04..BR-05
F-08	Messaging (Chat)	Must	BR-23
F-09	Artist/Studio Profile Claim	Must	BRL-11
F-10	Admin Panel & Audit	Must	BR-26..BR-29
F-11	Email & Notifications	Must	BR-24..BR-25
F-12	Observability & Analytics	Should	BR-30..BR-32
F-13	Compliance (PDPA)	Must	BR-10, BRL-13

4. External Interface Requirements

4.1 USER INTERFACES

- Design system:** Tailwind 4, CSS variables `--leish-header-from/to` (`#c9284b` family), `ThemeProvider` dark-by-default with inline anti-CLS script in `layout.tsx`.
- Pages** (`src/app/`): `/`, `/artists`, `/artists/[slug]`, `/artists/[slug]/book`, `/studios`, `/studios/[slug]`, `/login`, `/register`, `/forgot-password`, `/reset-password`, `/verify-email`, `/onboarding`, `/dashboard`, `/booking`, `/contact`, `/help`, `/privacy`, `/terms`, `/dev/emails` (dev only), `/admin/*`.
- Components** (`src/components/`): `Navbar`, `Footer`, `Button`, `ArtistCard`, `StudioCard`, `RatingStars`, `Logo`, `AdminShell`, `StatCard`, `Badge`, `TurnstileWidget`.
- Accessibility:** Keyboard navigation, focus-visible outlines, aria labels on filters/search, color contrast $\geq 4.5:1$, `prefer-reduced-motion` respected.
- Constraints:** No root `loading.tsx`; Suspense boundaries scoped to segments that don't call `notFound()`.

4.2 HARDWARE INTERFACES

None beyond standard web client/server. Server requires PG or local disk for SQLite (`data/leish.db` , WAL mode, `foreign_keys=ON`).

4.3 SOFTWARE INTERFACES

INTERFACE	PROTOCOL	MODULE	ENV VARS
PostgreSQL	<code>pg</code> Pool over TLS	<code>src/server/db.ts</code>	<code>DATABASE_URL</code> , <code>PG_MAX</code> , <code>PG_CONNECTION_TIMEOUT_MS</code> , <code>PG_IDLE_TIMEOUT_MS</code>
SQLite	<code>node:sqlite</code> DatabaseSync	<code>src/server/db.ts</code>	<code>LEISH_DB_PATH</code>
Billplz API v3	HTTPS POST <code>application/x-www-form-urlencoded</code> , Basic Auth	<code>src/server/payments.ts</code> , <code>src/lib/payments/billplz.ts</code>	<code>BILLPLZ_API_KEY</code> , <code>BILLPLZ_COLLECTION_ID</code> , <code>BILLPLZ_BASE_URL</code> , <code>BILLPLZ_X_SIGNATURE_KEY</code> , <code>BILLPLZ_CALLBACK_URL</code>
Billplz Webhook	HTTPS POST raw body, <code>X-Billplz-Signature</code> HMAC-SHA256 hex	<code>src/app/api/payments/webhook/route.ts</code>	same
Email — Resend	HTTPS JSON	<code>src/server/email.ts</code>	<code>EMAIL_PROVIDER=resend</code> , <code>RESEND_API_KEY</code> , <code>EMAIL_FROM</code>
Email — Postmark	HTTPS JSON	<code>src/server/email.ts</code>	<code>EMAIL_PROVIDER=postmark</code> , <code>POSTMARK_SERVER_TOKEN</code>
Email — Brevo	HTTPS JSON	<code>src/server/email.ts</code>	<code>EMAIL_PROVIDER=brevo</code> , <code>BREVO_API_KEY</code>
Supabase Storage	HTTPS (images)	<code>src/lib/supabase/*</code> , <code>next.config.ts</code> <code>images.remotePatterns</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>
Upstash Redis	REST	<code>src/server/rate-limit.ts</code>	<code>UPSTASH_REST_URL</code> , <code>UPSTASH_REST_TOKEN</code>
Turnstile	Widget + server verify	<code>src/components/TurnstileWidget</code> , <code>src/lib/turnstile-token.ts</code>	<code>NEXT_PUBLIC_TURNSTILE_SITE_KEY</code> , <code>TURNSTILE_SECRET_KEY</code>
Agnost AI	SDK + client track	<code>src/server/agnost.ts</code> , <code>src/lib/agnost-client.ts</code> , <code>src/instrumentation.ts</code>	<code>AGNOST_ORG_ID</code> , <code>NEXT_PUBLIC_AGNOST_ORG_ID</code>
Vercel Blob / S3	HTTPS	<code>src/app/api/upload/*</code>	<code>BLOB_READ_WRITE_TOKEN</code> , AWS S3 vars

4.4 COMMUNICATIONS INTERFACES

- **HTTP** only (HTTPS in prod). `Strict-Transport-Security` set via `next.config.ts`.
- **Security headers** (`next.config.ts`): `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, CSP with per-request nonce (no `unsafe-inline` for scripts except theme toggle).
- **Email**: via provider abstraction; outbox at `email_outbox` when `EMAIL_PROVIDER=dev`.
- **Chat SSE**: `GET /api/bookings/[id]/messages/stream` — `text/event-stream`, replay + live.
- **Health**: `GET /api/health` → 200 JSON.

5. System Features — Detailed (SRS-F)

Each feature lists its SRS requirements. Full functional decomposition with acceptance criteria lives in FRS (03).

5.1 SRS-F-01 — AUTHENTICATION & SESSION

ID	REQUIREMENT (SHALL)	VERIFICATION
SRS-F-01-01	Provide registration with fields <code>name</code> (2–80 chars), <code>email</code> (RFC), <code>password</code> (8–128 chars, script + optional <code>PEPPER_SECRET</code> HMAC), <code>role</code> in (<code>customer</code> , <code>artist</code> , <code>studio</code>), <code>consent</code> boolean + timestamp.	Unit (<code>src/server/validation.ts</code> <code>registerSchema</code>); e2e <code>register→login</code>
SRS-F-01-02	Hash passwords with <code>script</code> (salted, timing-safe compare); support <code>PEPPER_SECRET</code> pepper rotation via <code>PEPPER_VERSION</code> .	Auth tests; <code>src/server/auth.ts</code>
SRS-F-01-03	Issue HS256 JWT via <code>jose</code> , secret <code>SESSION_SECRET</code> (32-byte base64), payload includes <code>sub</code> , <code>role</code> , <code>jti</code> , <code>exp = now+7d</code> ; set <code>leish_session</code> cookie httpOnly, <code>SameSite=lax</code> , <code>Secure</code> in prod.	<code>src/server/session.ts</code> <code>createSessionToken</code>
SRS-F-01-04	Validate JWT on every authed route; check <code>sessions</code> table: <code>revoked=1</code> wins; missing JTI falls back to signature+expiry (fail-open, logged).	<code>verifySessionToken</code> tests
SRS-F-01-05	Revoke JTI on logout (<code>UPDATE sessions SET revoked=1 WHERE jti=?</code>).	<code>revokeSession</code> tests
SRS-F-01-06	Send single-use email verification link (token hashed sha256 at rest, configurable TTL), banner until <code>email_verified</code> ; gate payments until verified.	<code>src/server/email-verification.ts</code> ; <code>GET /api/auth/verify-email</code>
SRS-F-01-07	Provide forgot/reset password: single-use <code>password_resets</code> rows (hashed token, 1h expiry), rate-limited, no user enumeration (always 200-ish).	<code>src/app/api/auth/forgot-password/</code> *
SRS-F-01-08	Rate-limit auth endpoints (register/login/forgot/verify) with sliding window (Upstash or in-memory). Return 429 + <code>Retry-After</code> .	<code>src/server/rate-limit.ts</code>

5.2 SRS-F-02 — CATALOG

ID	REQUIREMENT	VERIFICATION
SRS-F-02-01	Store artists/studios in DB tables <code>artists</code> / <code>studios</code> (seeded from <code>src/lib/data.ts</code> via <code>src/server/catalog-seed.ts</code> idempotently; slugs == legacy ids).	<code>ensureCatalogSeeded()</code> ; <code>npm run db:seed-catalog</code>
SRS-F-02-02	Provide repository <code>listArtists(filters)</code> : SQL pre-filter <code>state/area/budget</code> (indexed), then <code>filterArtists()</code> for <code>query</code> /tags/date; <code>listAllArtists({limit,offset})</code> capped 500.	<code>src/server/catalog.ts</code>
SRS-F-02-03	Provide <code>getArtistById</code> , <code>getArtistBySlug</code> , <code>resolveArtist(idOrSlug)</code> (slug-first), same for studios.	Catalog tests
SRS-F-02-04	Validate discovery query via <code>artistsQuerySchema</code> (<code>MALAYSIA_STATES</code> , <code>BRIDAL_EVENTS</code> , <code>NON_BRIDAL_EVENTS</code>); support <code>state → area</code> cascading (<code>AREAS_BY_STATE</code>).	<code>src/server/validation.ts</code>
SRS-F-02-05	Allow admin direct column edits and artist self-service <code>PATCH /api/artist-profiles</code> (whitelisted fields only).	<code>updateArtist</code> / <code>updateStudio</code> field maps
SRS-F-02-06	Public catalog routes set <code>s-maxage=300</code> ; catalog pages are <code>force-dynamic</code> so edits surface without redeploy.	<code>src/app/artists/page.tsx</code> dynamic flag

5.3 SRS-F-03 — BOOKING MANAGEMENT

ID	REQUIREMENT	VERIFICATION
SRS-F-03-01	Create booking via <code>POST /api/bookings</code> — zod <code>bookingSchema</code> ; server resolves <code>price</code> from <code>artist.services.find(name)</code> — client price ignored.	Booking tests; <code>src/app/api/bookings/route.ts</code>
SRS-F-03-02	Record <code>event_type</code> , <code>venue</code> , <code>guest_count</code> , <code>notes</code> , <code>date</code> (ISO ≥ today), <code>time</code> string; initial <code>status='requested'</code> .	Schema + <code>bookings</code> table
SRS-F-03-03	Enforce slot uniqueness: partial unique index <code>uq_bookings_slot ON (artist_id,date,time) WHERE status IN (requested,accepted,confirmed)</code> ; friendly 409 on conflict (pre-check + race guard).	DB schema + booking route catch <code>UNIQUE</code>
SRS-F-03-04	Implement state machine <code>applyBookingTransition(current, action, {isOwner,role})</code> with rules per <code>src/server/bookings.ts:1</code> . Payment-only transition <code>confirmOnFeePaid</code> .	State machine unit tests
SRS-F-03-05	Expose <code>PATCH /api/bookings/[id]</code> for <code>accept/reject/complete/cancel</code> ; authz: <code>accept/reject/complete</code> requires claimed <code>artist / studio</code> ; <code>cancel</code> requires <code>isOwner</code> or artist.	Route tests
SRS-F-03-06	Serialize bookings with derived <code>quotation</code> , <code>totalPrice</code> , <code>balanceDueDate</code> (T-3d), <code>balanceAmount = total - bookingFeeSen</code> , <code>payment</code> + <code>balancePayment</code> .	<code>serializeBooking()</code>

5.4 SRS-F-04 — QUOTATIONS

ID	REQUIREMENT	VERIFICATION
SRS-F-04-01	Allow claimed artist to <code>POST /api/bookings/[id]/quotation</code> with <code>baseFee</code> , <code>travelFee</code> , <code>earlyCallFee</code> , <code>accommodationFee</code> , <code>extras[≤10]</code> , <code>artistNote</code> validated via <code>quotationSchema</code> .	<code>src/server/validation.ts</code>
SRS-F-04-02	Compute total server-side: <code>quotationTotal()</code> = sum of all line items (sen).	Unit test
SRS-F-04-03	Set <code>status='pending'</code> , <code>expires_at = now + 24h</code> (<code>QUOTATION_TTL_MS</code>); supersede previous pending (<code>superseded</code>).	<code>createQuotation()</code>
SRS-F-04-04	Lazily expire on read and via sweep: <code>isQuotationExpired()</code> , <code>findExpiredQuotations()</code> , <code>markQuotationExpired()</code> .	<code>src/server/quotations.ts</code>
SRS-F-04-05	Return 404 for already-expired quotations in booking serialization (<code>totalPrice → null</code> when expired).	<code>serializeBooking()</code>

5.5 SRS-F-05 — PAYMENTS

ID	REQUIREMENT	VERIFICATION
SRS-F-05-01	Provide hybrid payments: <code>payments.type IN ('deposit', 'balance') , UNIQUE(booking_id, type) , payments.status IN (required, paid, failed, refunded) .</code>	Schema + <code>src/server/payments.ts</code>
SRS-F-05-02	Create Billplz bill when <code>BILLPLZ_API_KEY</code> + <code>COLLECTION_ID</code> set, else <code>dev</code> payment row; body uses <code>amount</code> sen, <code>collection_id</code> , payer name/email from booking owner, <code>callback_url</code> , <code>redirect_url</code> , <code>reference_1=bookingId</code> .	<code>createBookingPayment()</code>
SRS-F-05-03	Expose <code>POST /api/bookings/[id]/pay-fee</code> (deposit, <code>getBookingFeeSen()</code>) and <code>POST /api/bookings/[id]/pay-balance</code> (quotation total – deposit); both authed, verified-email gated, dev auto-settles synchronously for e2e.	<code>pay-fee</code> / <code>pay-balance</code> routes
SRS-F-05-04	Handle webhook <code>POST /api/payments/webhook</code> — verify <code>X-Billplz-Signature</code> HMAC-SHA256 of raw body with <code>BILLPLZ_API_KEY</code> (timing-safe, hex64 check) before any state change.	<code>verifyBillplzSignature()</code>
SRS-F-05-05	Route settlement by type: <code>deposit</code> → <code>confirmOnFeePaid</code> then <code>UPDATE bookings SET status='confirmed' ; balance</code> → <code>UPDATE quotations SET status='paid'</code> + <code>createPayoutForBooking</code> ; idempotent.	<code>handlePaymentPaid()</code>
SRS-F-05-06	Allow refund of balance only on cancelled bookings when <code>status='paid'</code> ; <code>dev</code> marks refunded, <code>billplz</code> calls <code>POST /bills/{id}/refund</code> ; deposit never refunded.	<code>refundBalancePayment()</code>

5.6 SRS-F-06 — PAYOUTS & COMMISSION

ID	REQUIREMENT	VERIFICATION
SRS-F-06-01	Store platform settings <code>platform_settings(key,value)</code> for <code>booking_fee_sen</code> , <code>commission_rate_bps</code> (0–5000 clamp), <code>commission_waiver_sen</code> ; cached 30s.	<code>src/server/settings.ts</code>
SRS-F-06-02	Compute commission via <code>computeCommission(totalSen, rateBps, waiverSen)</code> — waived when <code>total < waiver</code> ; <code>commission = round(total*rate/10000)</code> .	Unit tests
SRS-F-06-03	On balance paid, create <code>payouts</code> row: <code>gross=total</code> , <code>commission</code> , <code>net = max(0, (total - commission) - deposit)</code> , <code>settleable_at = eventDate+24h</code> , <code>status='pending'</code> ; idempotent per <code>booking_id</code> .	<code>createPayoutForBooking()</code>
SRS-F-06-04	Provide <code>listPayouts(status?)</code> joined with bookings, and <code>updatePayoutStatus(id, settled failed, notes)</code> with <code>settled_at</code> when applicable; admin-only.	<code>src/server/payouts.ts</code> + <code>GET/PATCH /api/admin/payouts</code>

5.7 SRS-F-07 — REVIEWS & RATINGS

ID	REQUIREMENT	VERIFICATION
SRS-F-07-01	Allow adding review via <code>addEntityReview({entityType,entityId,bookingId,userId,authorName,rating 1-5,event,text})</code> ; guard <code>ALREADY_REVIEWED</code> when <code>booking_id</code> duplicate.	<code>src/server/catalog.ts</code> reviews section
SRS-F-07-02	Gate eligibility: <code>findReviewableBooking(userId, artistId)</code> returns most recent <code>status='completed'</code> booking left-joined against <code>reviews</code> (null → not yet reviewed). Studios not yet reviewable (returns null).	Query test
SRS-F-07-03	Blend rating atomically: <code>UPDATE artists SET review_count=review_count+1, rating=ROUND(((rating*review_count)+:rating)/(review_count+1),2)</code> — single statement, no read-modify-write.	<code>blendAggregate()</code>

5.8 SRS-F-08 — MESSAGING

ID	REQUIREMENT	VERIFICATION
SRS-F-08-01	Store <code>messages(id, booking_id FK, sender_id FK, body, created_at)</code> per-booking.	Schema
SRS-F-08-02	Expose <code>GET/POST /api/bookings/[id]/messages</code> (participants only) and <code>GET .../messages/stream</code> SSE (replay history then live via <code>src/server/chat-bus.ts</code>).	Route + e2e

5.9 SRS-F-09 — PROFILE CLAIM

ID	REQUIREMENT	VERIFICATION
SRS-F-09-01	Provide <code>artist_profiles(user_id PK, artist_id UNIQUE, claimed_at)</code> and <code>studio_profiles</code> — one claim per account.	Schema + <code>uq_*_profiles_*</code> indexes
SRS-F-09-02	Expose <code>GET /api/artist-profiles</code> (own profile or null), <code>POST /api/artist-profiles {artistId}</code> (scopes bookings), <code>PATCH /api/artist-profiles</code> (whitelisted self-edits).	<code>src/server/artist-profiles.ts</code>

5.10 SRS-F-10 — ADMIN PANEL & AUDIT

ID	REQUIREMENT	VERIFICATION
SRS-F-10-01	Guard all <code>src/app/api/admin/*</code> via <code>requireAdmin(request)</code> (401 anonymous, 403 non-admin).	<code>src/server/admin-auth.ts</code>
SRS-F-10-02	Guard <code>/admin/*</code> layout server-side redirect for non-admins.	<code>src/app/admin/layout.tsx</code>
SRS-F-10-03	Provide admin APIs: <code>GET /api/admin</code> dashboard aggregates, <code>/api/admin/users</code> CRUD, <code>/api/admin/artists</code> , <code>/api/admin/studios</code> , <code>/api/admin/bookings</code> , <code>/api/admin/payments</code> , <code>/api/admin/payouts</code> , <code>/api/admin/quotations</code> , <code>/api/admin/messages</code> , <code>/api/admin/emails</code> , <code>/api/admin/audit</code> , <code>/api/admin/settings</code> , <code>/api/admin/analytics</code> .	<code>src/app/api/admin/route.ts</code> listing test
SRS-F-10-04	Log every admin mutation to <code>admin_audit_log(id, admin_user_id FK, action, target_table, target_id, details JSON, created_at)</code> via <code>logAdminAction()</code> .	Audit table + tests
SRS-F-10-05	Allow seeding first admin idempotently: <code>ADMIN_EMAIL/ADMIN_PASSWORD npx tsx scripts/seed-admin.ts</code> upgrades existing users.	<code>scripts/seed-admin.ts</code>

5.11 SRS-F-11 — EMAIL & NOTIFICATIONS

ID	REQUIREMENT	VERIFICATION
SRS-F-11-01	Abstract <code>sendEmail()</code> over <code>EMAIL_PROVIDER=dev resend postmark brevo</code> ; missing creds fall back to dev outbox with warning (never silent).	<code>src/server/email.ts</code>
SRS-F-11-02	Compose booking emails in <code>src/server/booking-emails.ts</code> (created, quotation, balance reminder, status).	Module
SRS-F-11-03	Store <code>email_outbox</code> (dev), <code>email_preferences</code> per user (flags per event), <code>email_retries</code> (attempts ≤ 3 , <code>next_retry</code>).	Schema
SRS-F-11-04	Expose <code>/dev/emails</code> viewer (dev only) and <code>/api/email/preferences</code> toggle.	Routes

5.12 SRS-F-12 — OBSERVABILITY & ANALYTICS

ID	REQUIREMENT	VERIFICATION
SRS-F-12-01	Log with <code>pino</code> (<code>src/server/logger.ts</code>): JSON in prod, pretty in dev; <code>LOG_LEVEL</code> override; forward batches (20 lines / 50ms debounce) to <code>LOG_WEBHOOK_URL</code> .	Logger tests
SRS-F-12-02	Report errors via <code>reportError(err, context)</code> → pino + Sentry envelope if <code>SENTRY_DSN</code> else <code>ERROR_WEBHOOK_URL</code> .	<code>src/server/errors.ts</code>
SRS-F-12-03	Instrument with Agnost AI (<code>src/instrumentation.ts</code> <code>agnostai</code> init): <code>agnost.begin({userId, agentName, input})</code> → <code>interaction.end(output, success)</code> on register/login/booking/quotation/payment/profile-edit. Client: <code>trackEvent/trackArtistView/trackSearch/trackBookingForm</code> (<code>src/lib/agnost-client.ts</code>).	Agnost dashboard <code>3e27e121-654d-4746-ba55-7996f21bb351</code>

5.13 SRS-F-13 — COMPLIANCE

ID	REQUIREMENT	VERIFICATION
SRS-F-13-01	Implement <code>GET /api/me/export</code> (own data JSON) and <code>DELETE /api/me?confirm=1</code> (cascade delete via FKs).	<code>src/app/api/me/*</code>
SRS-F-13-02	Serve <code>/privacy</code> and <code>/terms</code> with templates from <code>docs/legal/*</code> .	Pages
SRS-F-13-03	Health check <code>GET /api/health</code> returns DB + env status.	<code>src/app/api/health/*</code> + Docker <code>HEALTHCHECK</code>

6. Data Requirements

6.1 LOGICAL DATA MODEL (SUMMARY)

See dedicated `docs/06-Data-Model-and-ERD.md` for full DDL, ER diagram, and field dictionaries. Summary here:

ENTITY GROUP	TABLES	PK/FK	NOTES
Users	users	PK id	role CHECK (customer artist studio admin); consent+consent_timestamp; script password
Catalog	artists, studios, reviews, referrals	PK id, slug UNIQUE for catalog	rating float, review_count, JSON arrays (specialties, services, etc.)
Claims	artist_profiles, studio_profiles	PK user_id → users, UNIQUE on artist_id / studio_id	Scopes bookings per actor
Booking	bookings	PK id, FK user_id → users, partial UNIQUE slot index	uq_bookings_slot WHERE active statuses
Quotation	quotations	PK id, FK booking_id → bookings ON DELETE CASCADE	Fee columns sen, extras JSON
Payments	payments, payouts, platform_settings	UNIQUE(booking_id, type) for payments	status CHECKs; computeCommission drives payouts
Comms	messages, email_outbox, email_preferences, email_retries, email_verifications, password_resets	FKs to users/bookings	Preference flags per event
Ops	sessions, admin_audit_log, catalog_overrides (legacy), platform_settings	FKs	Audit FK to users(id); tests must seed user row first

All tables include created_at (ISO string); updated_at where mutation is expected. Money columns are INTEGER sen.

6.2 DATA RETENTION

Per docs/PDPA_RETENTION_GUIDELINES.md: users/bookings retained for audit period after account deletion where legal hold exists; otherwise cascade delete. Quotations/payments retained ≥7 years for financial audit (configurable).

6.3 DATA INTEGRITY

- Foreign keys enforced (PRAGMA foreign_keys=ON for SQLite, real FKs for PG).
- Placeholder translation ?/@name → \$n tested in src/server/db.ts (compilePlaceholders, resolveParams).
- Drift detection warns when PG_SCHEMA and SQLITE_SCHEMA diverge.

7. Other Requirements

7.1 PERFORMANCE (TARGETS — SEE NFR DOC FOR DETAIL)

METRIC	TARGET
Catalog list (p50)	<250 ms (p95 <600 ms) via SQL pre-filter + index
Booking creation (p95)	<400 ms excluding email send (async)
Webhook handling (p95)	<300 ms
Image delivery	Next.js optimized, <code>*.supabase.co</code> via proxy, CDN

7.2 SAFETY & SECURITY (SUMMARY)

- Passwords: scrypt + pepper, never logged.
- Sessions: httpOnly, secure in prod, JTI revocation.
- Inputs: zod on every API boundary.
- Headers: CSP nonce, `DENY`, `nosniff`, `strict-origin-when-cross-origin`.
- Rate limiting: sliding window, 429.
- Audit: every admin mutation logged.

7.3 SOFTWARE SYSTEM ATTRIBUTES — SUMMARY

Quality attributes are elaborated in `docs/04-NFR-Non-Functional-Requirements.md` (availability, maintainability, portability, etc.).

8. Verification & Validation

8.1 TEST STRATEGY

LEVEL	TOOL	LOCATION	GATE
Unit	Vitest (jsdom)	<code>src/**/*.test.{ts,tsx}</code> (exclude <code>src/lib/data.ts</code>)	All green
Component	Testing Library	<code>src/components/**/*.test.tsx</code>	Green
Integration	Vitest with <code>LEISH_DB_PATH=:memory:</code> SQLite	<code>src/server/**/*.test.ts</code> , API route tests	Green
E2E	Playwright (<code>chromium</code>)	<code>e2e/</code>	Green; browser installed via <code>playwright install --with- deps chromium</code>
Contract	Zod validation tests	<code>src/server/validation.test.ts</code>	Green
Security	ESLint, <code>npm audit</code> , rate- limit tests	CI	0 warnings
Quality Gate	<code>quality-gate.yml</code> : lint, typecheck, test, coverage ≥80%, format	CI	Required per PR
DB Dual Path	<code>vitest.pg.config.mts</code> when <code>DATABASE_URL</code> set	CI <code>database.yml</code> when <code>db.ts migrate.ts</code> changes	Green

8.2 ACCEPTANCE TESTS (TRACE TO SRS)

SRS-F	E2E / TEST SCENARIO
F-01	Register with role → receive verification banner → resend → verify → login → 7d session persists across refresh
F-02	Browse <code>/artists</code> unauthenticated → filter state→area, bridal tag, budget, query → detail page resolves by slug
F-03	Customer creates booking for future date → second concurrent request to same slot returns 409 → artist accepts
F-04	Artist sends quotation → customer sees 24h countdown → expiry auto-marks + re-quote flow
F-05	Customer pays deposit (dev → auto-confirm) → status <code>confirmed</code> → pays balance → payout created
F-07	After <code>completed</code> , customer leaves review → second review for same booking rejected → rating rounded 2dp
F-08	Chat SSE receives history + live messages; only participants can post
F-10	Non-admin blocked from <code>/admin</code> + <code>/api/admin/*</code> ; admin actions audited
F-11	Verification email lands in <code>/dev/emails</code> when provider=dev
F-13	Export returns owned data; delete removes account and cascades bookings

9. Appendices

9.1 FEASIBILITY

Technical feasibility confirmed on Node 22 + pg 8.23 + jose 6.2.8 + pino 10.3. Tested locally via SQLite and in CI via PG. Billplz sandbox tested manually. No blocking unknowns.

9.2 FUTURE ENHANCEMENTS (BACKLOG)

- Studio-native `studio_id` booking path (Option B already scaffolded in `bookings` table) with distinct commission tiers.
- Product catalog (`products` type/table `src/lib/types.ts:89`) with cart/checkout.
- Referral rewards (`referrals` table) activation and commission share.
- WhatsApp notifications, advanced search (Vectorize), promotions engine.

9.3 DOCUMENT CHANGE HISTORY

VERSION	DATE	CHANGE
1.0	2026-08-15	Initial SRS — booking→fee single-path
2.0	2026-08-29	Hybrid deposit/balance, payouts, messaging, admin audit, Agnost

Next: [03-FRS-Functional-Requirements-Specification.md](#) details every FR with actor, pre/post conditions, flow, and acceptance criteria.